1- Introduction aux réseaux de neurones convolutifs

installer un package

```
# !pip install -q tensorflow
```

1- Introduction aux réseaux de neurones convolutifs

installer un package

```
# !pip install -q tensorflow
```

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D
```

```
image = data.camera()

plt.imshow(image, cmap='gray')
plt.title("Image originale")
plt.axis('off')
plt.show()
```

Image originale

1- Introduction aux réseaux de neurones convolutifs

```
image_cnn = np.expand_dims(np.expand_dims(image, axis=0), axis=-1)
```

```
kernel = np.array([
    [-1, -1, -1],
    [-1,  8, -1],
    [-1, -1, -1]
], dtype=np.float32).reshape(3, 3, 1, 1)

model = Sequential()
model.add(Conv2D(
    1,
    (3,3),
    strides=1,
    use_bias=False,
    input_shape=(image.shape[0], image.shape[1], 1)
))

model.layers[0].set_weights([kernel])

result = model.predict(image_cnn)

plt.imshow(result[0, :, :, 0], cmap='gray')
plt.title("Filtre contours - stride 1")
plt.axis('off')
plt.show()

result.shape
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step
```

Filtre contours - stride 1

```
(1, 510, 510, 1)
```

1- Introduction aux réseaux de neurones convolutifs

```
sobel_vertical = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
], dtype=np.float32)
```

```
sobel_horizontal = np.array([
    [-1, -2, -1],
    [ 0,  0,  0],
    [ 1,  2,  1]
], dtype=np.float32)
```

```
blur = np.ones((3,3),
dtype=np.float32) / 9
```

```
identity = np.array([
    [0, 0, 0],
    [0, 1, 0],
    [0, 0, 0]
], dtype=np.float32)
```

1- Introduction aux réseaux de neurones convolutifs

Exemple de CNN générique

```python
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# 1. Charger les données MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# 2. Redimensionner
x_train = x_train.reshape(-1,28,28,1)
x_test = x_test.reshape(-1,28,28,1)

# 3. Créer le modèle CNN
model = Sequential()
model.add(Conv2D(8, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D((2,2)))  # réduire la taille
model.add(Flatten())
model.add(Dense(10, activation='softmax'))  # 10 classes

# 4. Compiler
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# 5. Entraîner
model.fit(x_train, y_train, epochs=3, batch_size=32)

# 6. Évaluer
loss, acc = model.evaluate(x_test, y_test)
print("Précision test:", acc)
```

1- Introduction aux réseaux de neurones convolutifs

Exemple de CNN générique

```python
import matplotlib.pyplot as plt
import numpy as np

# On prend la première image du test set
image = x_test[0]  # MNIST test set
label_reel = y_test[0]

# Visualiser l'image
plt.imshow(image.reshape(28,28), cmap='gray')
plt.title(f"Label réel : {label_reel}")
plt.show()

# Préparer pour le modèle
image_input = image.reshape(1,28,28,1)  # ajouter batch
prediction = model.predict(image_input)
classe_predite = np.argmax(prediction)

print("Classe prédite :", classe_predite)
```